

# Maximilian Angular the Complete Guide 2025 1

## 1-Essentials Signals input, output, computed

```
1 import { Component } from '@angular/core';
2
3 import { HeaderComponent } from './header/header.component';
4 import { UserComponent } from './user/user.component';
5 import { TasksComponent } from './tasks/tasks.component';
6
7 import { DUMMY_USERS } from './dummy-users';
8
9 @Component({
10   selector: 'app-root',
11   standalone: true,
12   templateUrl: './app.component.html',
13   styleUrls: ['./app.component.css'],
14   imports: [HeaderComponent, UserComponent, TasksComponent],
15 })
16 export class AppComponent {
17   users = DUMMY_USERS;
18   selectedUserId?: string;
19
20   get selectedUser() {
21     return this.users.find((user) => user.id === this.selectedUserId);
22   }
23
24   onSelectUser(id: string) {
25     this.selectedUserId = id;
26   }
27 }
```

```
1 import { Component, input, computed, output } from '@angular/core';
2
3 import { type User } from './user.model';
4 import { CardComponent } from '../shared/card/card.component';
5
6 @Component({
7   selector: 'app-user',
8   standalone: true,
9   templateUrl: './user.component.html',
10   styleUrls: ['./user.component.css'],
11   imports: [CardComponent],
12 })
13 export class UserComponent {
14   user = input.required<User>();
15   selected = input.required<boolean>();
16
17   imagePath = computed(() => 'assets/users/' + this.user().avatar);
18
19   select = output<string>();
20
21   onSelectUser() {
22     this.select.emit(this.user().id);
23   }
24 }
```

```
1 <app-header />
2
3 <main>
4   <ul id="users">
5     @for (user of users; track user.id) {
6       <li>
7         <app-user
8           [user]="user"
9           [selected]="user.id === selectedUserId"
10           (select)="onSelectUser($event)"
11         />
12       </li>
13     }
14   </ul>
15
16   @if (selectedUser) {
17     <app-tasks [userId]="selectedUser.id" [name]="selectedUser.name" />
18   } @else {
19     <p id="fallback">Select a user to see their tasks!</p>
20   }
21 </main>
```

```
1 <app-card>
2   <button [class.active]="selected()" (click)="onSelectUser()">
3     <img [src]="imagePath()" [alt]="user().name" />
4     <span>{{ user().name }}</span>
5   </button>
6 </app-card>
```

**<app-card>** üzerinde ki **(click)="onSelectUser()"** ifadesi **<button>** üzerinde ki click event'ini tetikler ve **UserComponent** sınıfındaki **onSelectUser** metodunu çalıştırır.

**UserComponent.onSelectUser()** metodu **this.select.emit(this.user().id)** komutunu işleterek select output signal'inin fırlatılarak(emit) komponent dışına yayınlanmasını sağlar.

Component dışına fırlatılan **select** output signal'i **<app-header/>** altında oluşturulan **<app-user/>** tarafından **(select)="onSelectUser(\$event)"** ifadesiyle yakalanır ve AppComponent.onSelectUser(id:string) metodu çalıştırılarak seçili kullanıcı app.ts componenti üzerinde tutulur.

Component'ler üzerinde kullanılan signal yapıları component'in hayat döngüsü içinde varlık gösterir ve üzerinde barındırdığı veride bir değişiklik olduğunda bu değişikliği bulunduğu component'e yansıtır. UserComponent sınıfı içinde bulunan **input**, **computed** ve **output** metodları signal kullanımındır.

## 1-Essentials Signals

## header

```

1 import { Component, input, inject, signal, output } from '@angular/core';
2 import { FormsModule } from '@angular/forms';
3
4 import { TasksService } from '../tasks.service';
5
6 @Component({
7   selector: 'app-new-task',
8   standalone: true,
9   imports: [FormsModule],
10  templateUrl: './new-task.component.html',
11  styleUrls: ['./new-task.component.css'],
12 })
13 export class NewTaskComponent {
14   close = output<void>();
15
16   userId = input.required<string>();
17
18   enteredTitle = signal('');
19   enteredSummary = signal('');
20   enteredDate = signal('');
21
22   private tasksService = inject(TasksService);
23
24   onCancel() {
25     this.close.emit();
26   }
27
28   onSubmit() {
29     this.tasksService.addTask(
30       {
31         title: this.enteredTitle(),
32         summary: this.enteredSummary(),
33         date: this.enteredDate(),
34       },
35       this.userId()
36     );
37     this.close.emit();
38   }
39 }

```

**new-task.ts**  
**NewTaskComponet{}**

Form işlemlerinde çift taraflı binding yapabilecek signal kullanımı örneğini inceleyelim;

NewTaskComponent sınıfı içinde new-task.html üzerindeki formda kullanılacak çift taraflı binding yapacak form input elemanları signal("") fonksiyonuyla tanımlanır. Form üzerinde bu signal metodları **[(ngSubmit)]** ifadesiyle çift taraflı olduğu belirtilir.

```

1 @if (isAddingTask) {
2   <app-new-task [userId]="userId()" (close)="onCloseAddTask()" />
3 }
4
5 <section id="tasks">
6   <header>
7     <h2>{{ name() }}'s Tasks</h2>
8     <menu>
9       <button (click)="onStartAddTask()">Add Task</button>
10    </menu>
11  </header>
12
13  <ul>
14    <for (task of selectedUserTasks; track task.id) {
15      <li>
16        <app-task [task]="task" />
17      </li>
18    }
19  </ul>
20 </section>

```

**tasks.html**

```

1 <div class="backdrop" (click)="onCancel()"></div>
2 <dialog open>
3   <h2>Add Task</h2>
4   <form [(ngSubmit)="onSubmit()">
5     <p>
6       <label for="title">Title</label>
7       <input type="text" id="title" name="title" [(ngModel)]="enteredTitle" />
8     </p>
9
10    <p>
11      <label for="summary">Summary</label>
12      <textarea
13        id="summary"
14        rows="5"
15        name="summary"
16        [(ngModel)]="enteredSummary"
17      ></textarea>
18    </p>
19
20    <p>
21      <label for="due-date">Due Date</label>
22      <input
23        type="date"
24        id="due-date"
25        name="due-date"
26        [(ngModel)]="enteredDate"
27      />
28    </p>
29
30    <p class="actions">
31      <button type="button" (click)="onCancel()">Cancel</button>
32      <button type="submit">Create</button>
33    </p>
34  </form>
35 </dialog>

```

**new-task.html**